

Asynchronous Streams in C# 8 with Examples

In this article, I am going to discuss **Asynchronous Streams in C# 8** with Examples. Please read our previous article where we discussed [Nullable Reference Types in C# 8](#) with Examples.

Introduction to Asynchronous Streams

Async Streams is the new feature in C# 8.0 which provides async support for handling streams or IEnumerable data. In this article, we cover all the aspects of Async Streams (IAsyncEnumerable) including how to use ConfigureAwait and how to use CancellationToken as well. Cancellation token can be a great way to manage async programming in Dotnet core and C# but with Async Streams, it can be a bit difficult and at times if not used properly can give errors. We shall be discussing briefly everything about AsyncStreams. IAsyncEnumerable also has options and support for CancellationToken as well as ConfigureAwait.

Asynchronous Streams in C#

Starting from C# 8.0, you can create and consume streams asynchronously. A method that returns an asynchronous stream has three properties:

1. It's declared with the async modifier.
2. It returns an IAsyncEnumerable<T>.
3. The method contains yield return statements to return successive elements in the asynchronous stream.

Consuming an asynchronous stream requires you to add the await keyword before the foreach keyword when you enumerate the elements of the stream. Adding the await keyword requires the method that enumerates the asynchronous stream to be declared with the async modifier and to return a type allowed for an async method. Typically, that means returning a Task or Task<TResult>. It can also be a ValueTask or ValueTask<TResult>. A method can both consume and produce an asynchronous stream, which means it would return an IAsyncEnumerable<T>. The following code generates a sequence from 0 to 19, waiting 100 ms between generating each number.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    public class NullableReferenceTypes
    {
        static async Task Main(string[] args)
        {
            await foreach (var number in GenerateSequence())
            {
                Console.WriteLine(number);
            }
        }
        public static async System.Collections.Generic.IAsyncEnumerable<int>
GenerateSequence()
        {
            for (int i = 0; i < 20; i++)
```

```

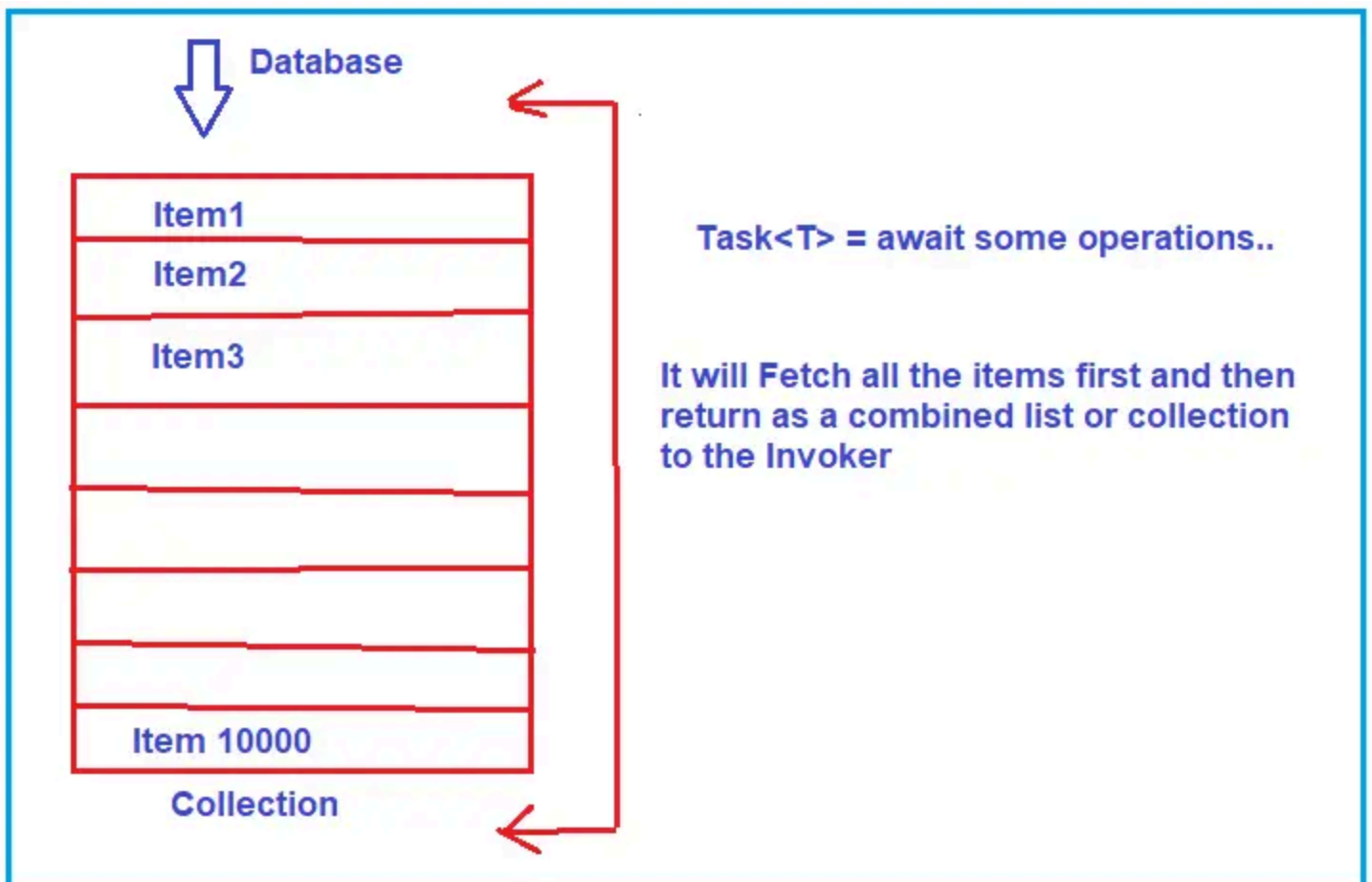
    {
        await Task.Delay(100);
        yield return i;
    }
}
}

```

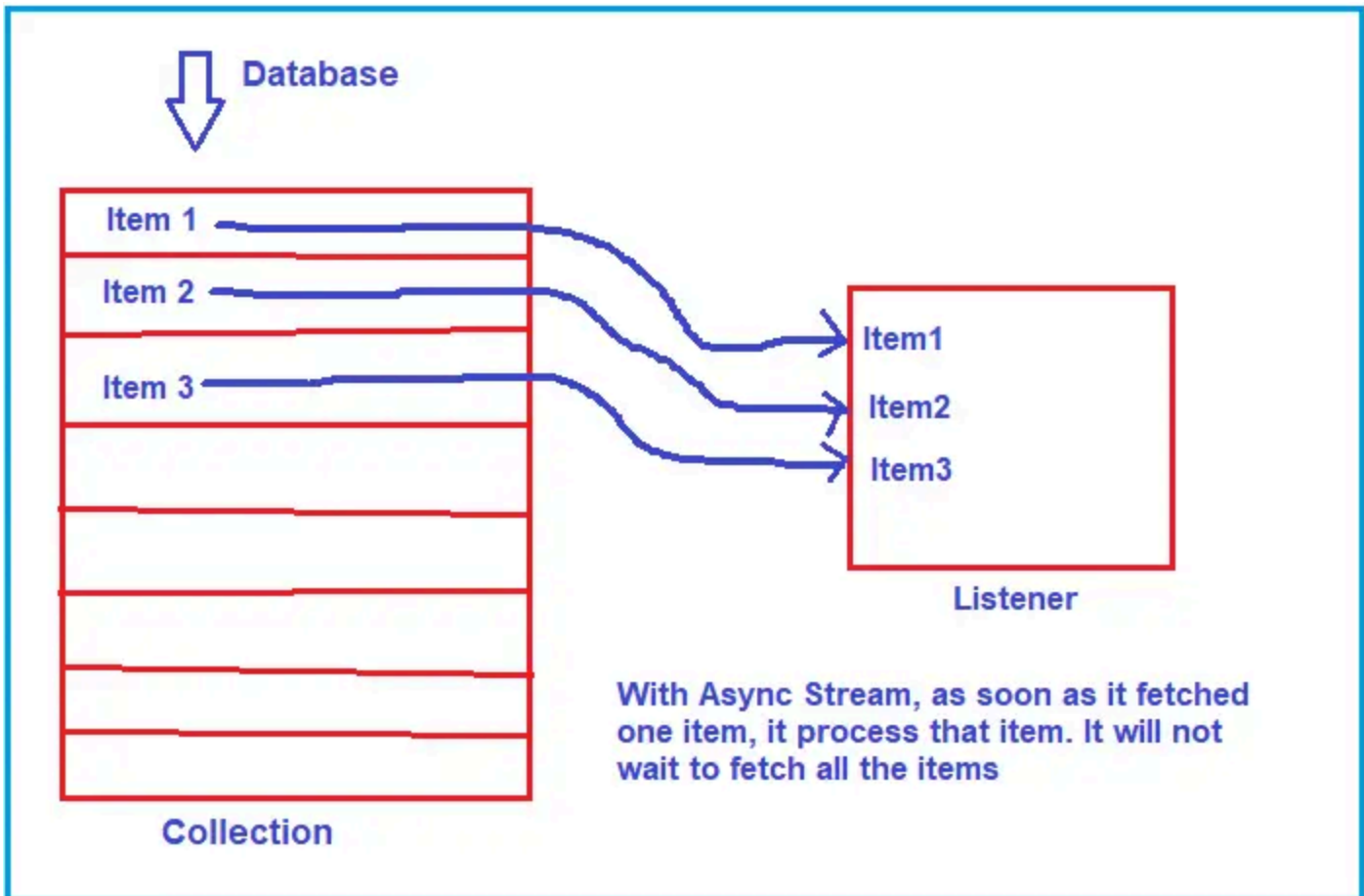
You can observe in the above code, the method `GenerateSequence` is declared with the `async` keyword and the return type is `IAsyncEnumerable<T>` i.e. `IAsyncEnumerable<int>`. The `yield` method also contains the `yield return` statements to return successive elements in the asynchronous stream. Then we are using here the `async` main method to call the `async` `GenerateSequence` method, Further, notice here we enumerate the sequence using the `await foreach` statement. Now, let us proceed and understand this concept in detail.

Generate and Consume Async Streams using C# 8.0 and .NET Core 3.0:

Let's say we have a collection which different data items. Let us also assume that the collection items come from a database. Generally, what we do is use `Task<T>` and `async` and `await` mechanism to read the collection. Then what will happen internally is, that these `async` and `await` operator will fetch all the data first and then only sends those data to the called as a combined list or collection. For example, if the collection has 10000 items, then all those 10000 items will fetch in-memory first and then all those records will return as a combined list or combined collection to the invoker. For a better understanding, please have a look at the below image.



Let's say the time taken to fetch all the records is 5 seconds, then I have to wait for five seconds. But the thing is quite different when we try to use async stream or `IAsyncEnumerable`. So, the `IAsyncEnumerable` is the base interface that we return in Async Streaming. Let us say the database return 10000 items i.e. the collection has 10000 items. In this case, as the item comes then will move back to the original invoker. They will not wait to fetch all the records in memory. Let us say there is a listener who is listening to these records. So, what will happen is, that as soon as the first record comes, it will send out to the listener. Similarly, when the second record comes, it will again send to the listener, and so on. So, I am not waiting here for all the records to be fetched. Now, I will not wait 5 seconds to receive the data. As soon as the data is received, it processes parallelly. For a better understanding, please have a look at the below image.



So, with Async Stream, it will start processing the data as soon as it is available rather than waiting till the end. This can drastically improve the application performance and overall user experience in many cases.

Example to Understand Asynchronous Stream in C#:

Let us understand Async Stream in C#. First, we will see one example without using Async Stream and then we will see how to convert this example to an async stream.

Please have a look at the below code. In the below example, the `GetNumbers` method simply generates numbers and returns those numbers as a list to the invoker. While generating the number we make the code sleep between 500 to 1500 milliseconds randomly. From the main method, we are just calling the `GetNumbers` method and also printing the start and end time to see how much time the `GetNumbers` method took to generate 10 numbers.

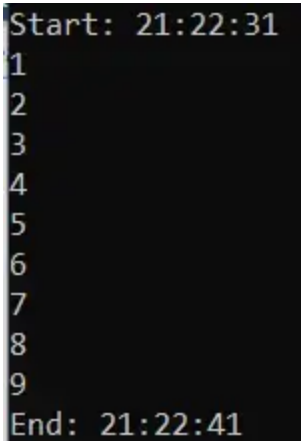
```
using System;  
using System.Collections.Generic;
```

```

using System.Threading;
namespace Csharp8Features
{
    public class AsynchronousStreams
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Start: {DateTime.Now.ToLongTimeString()}");
            var numbers = GetNumbers(1, 10);
            Console.WriteLine($"End: {DateTime.Now.ToLongTimeString()}");
            Console.ReadKey();
        }
        public static IEnumerable<int> GetNumbers(int start, int end)
        {
            var random = new Random();
            var returnList = new List<int>();
            for (int i = start; i < end; i++)
            {
                returnList.Add(i);
                Thread.Sleep(millisecondsTimeout: random.Next(500, 1500));
            }
            return returnList;
        }
    }
}

```

Output:



```

Start: 21:22:31
1
2
3
4
5
6
7
8
9
End: 21:22:41

```

You can see it took approximately 10 seconds to process this.

Example using Asynchronous Stream in C#:

Let us rewrite the previous example using Async Stream. Here, we created the `GetNumbersAsync` method with the `async` keyword and also, we use `IAsyncEnumerable<int>` as the return type. The logic is going to be the same, but here instead of creating a collection and storing the numbers in that collection, what we are doing is, we are using `yield` to return the generated number. This will asynchronously return the number to the invoker i.e. to the main method and then wait for the move next method of the enumerator. In the main

method, first, we change the main to async main. Then we are calling the GetNumbersAsync(1, 10) method to generate 10 numbers. And here we are using an await foreach loop to print the numbers on the console.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
namespace Csharp8Features
{
    public class AsynchronousStreams
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine($"Start: {DateTime.Now.ToLongTimeString()}");
            var numbersAsync = GetNumbersAsync(1, 10);
            await foreach (var number in numbersAsync)
            {
                Console.WriteLine(number);
            }
            Console.WriteLine($"End: {DateTime.Now.ToLongTimeString()}");
            Console.ReadKey();
        }
        public static async IEnumerable<int> GetNumbersAsync(int start, int end)
        {
            var random = new Random();

            for (int i = start; i < end; i++)
            {
                await Task.Delay(random.Next(500, 1500));
                yield return i;
            }
        }
    }
}
```

Now run the above code and see how much time it is taking to process the data. Further, if you notice, it will not wait to process all the records, it will process the record asynchronously. See, it took 8 seconds in my machine. It might be different in your machine. Again, if you run the code, then you might get a different time. But, important point is, that you will get the data asynchronously.

```
Start: 21:40:17
1
2
3
4
5
6
7
8
9
End: 21:40:25
```

While working with Async Stream, we need to understand two more things i.e. configure await and cancellation token. Let us understand these two points with examples.

Cancellation Token in C#:

The Cancellation Token is basically used to cancel operations. For example, we want the GetNumbersAsync method to be cancelled its execution after 3000 milliseconds. In that case, we need to use Cancellation Token. Let's see how to use the Cancellation token in C#. First, we need to modify the GetNumbersAsync method as follows. First, we are adding the CancellationToken as a parameter and this is used to cancel the enumeration and that's why we need to decorate this parameter with the [EnumeratorCancellation] type. Then we are checking the IsCancellationRequested property and if its value is true then we are cancelling the execution of this method by using the break statement.

```
public static async IAsyncEnumerable<int> GetNumbersAsync(int start, int end,
[EnumeratorCancellation] CancellationToken token = default)
{
    var random = new Random();
    for (int i = start; i < end; i++)
    {
        if (token.IsCancellationRequested)
        {
            Console.WriteLine("Cancellation has been requested...");
            // Perform cleanup if necessary.
            //...
            // Terminate the operation.
            break;
        }
        await Task.Delay(random.Next(500, 1500));
        yield return i;
    }
}
```

Then while calling the GetNumbersAsync method, we need to create an instance of CancellationToken and we need to set the time when the execution of the method is going to be cancelled. Please have a look at the below code.

First, we are creating an instance of CancellationTokenSource (var cancellationTokenSource = new CancellationTokenSource()) and then we are setting time when it is going to be cancelled (cancellationTokenSource.CancelAfter(millisecondsDelay: 3000)) and then we are passing the Cancellation token as a parameter to the GetNumbersAsync method (var numbersAsync = GetNumbersAsync(1, 10, cancellationTokenSource.Token))

```
var cancellationTokenSource = new CancellationTokenSource();
cancellationTokenSource.CancelAfter(millisecondsDelay: 3000);
var numbersAsync = GetNumbersAsync(1, 10, cancellationTokenSource.Token);
```

Complete Code Example showing the use of Cancellation Token in C#:

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
```

```

using System.Threading;
using System.Threading.Tasks;
namespace Csharp8Features
{
    public class AsynchronousStreams
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine($"Start: {DateTime.Now.ToLongTimeString()}");
            var cancellationTokenSource = new CancellationTokenSource();
            cancellationTokenSource.CancelAfter(millisecondsDelay:3000);
            var numbersAsync = GetNumbersAsync(1, 10, cancellationTokenSource);
            await foreach (var number in numbersAsync)
            {
                Console.WriteLine(number);
            }
            Console.WriteLine($"End: {DateTime.Now.ToLongTimeString()}");
            Console.ReadKey();
        }

        public static async IEnumerable<int> GetNumbersAsync(int start, int end,
[EnumeratorCancellation]CancellationToken token = default)
        {
            var random = new Random();

            for (int i = start; i < end; i++)
            {
                if (token.IsCancellationRequested)
                {
                    Console.WriteLine("Cancellation has been requested...");
                    // Perform cleanup if necessary.
                    //...
                    // Terminate the operation.
                    break;
                }
                await Task.Delay(random.Next(500, 1500));
                yield return i;
            }
        }
    }
}

```

Output:

```

Start: 22:30:12
1
2
3
Cancellation has been requested...
End: 22:30:16

```

ConfigureAwait Method in C#:

The ConfigureAwait method Configures how awaits on the tasks returned from an async iteration are performed. true to capture and marshal back to the current context; otherwise, false. Let us understand this with an example.

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Threading;
using System.Threading.Tasks;
namespace Csharp8Features
{
    public class AsynchronousStreams
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine($"Start: {DateTime.Now.ToLongTimeString()}");
            var cancellationTokenSource = new CancellationTokencancellationTokenSource();
            cancellationTokenSource.CancelAfter(millisecondsDelay:3000);
            var numbersAsync = GetNumbersAsync(1, 10,
cancellationTokenSource.Token).ConfigureAwait(false);
            await foreach (var number in numbersAsync)
            {
                Console.WriteLine(number);
            }
            Console.WriteLine($"End: {DateTime.Now.ToLongTimeString()}");
            Console.ReadKey();
        }
        public static async IAsyncEnumerable<int> GetNumbersAsync(int start, i
[EnumeratorCancellation]CancellationTokencancellationToken token = default)
        {
            var random = new Random();

            for (int i = start; i < end; i++)
            {
                if (token.IsCancellationRequested)
                {
                    Console.WriteLine("Cancellation has been requested...");
                    // Perform cleanup if necessary.
                    //...
                    // Terminate the operation.
                    break;
                }
                await Task.Delay(random.Next(500, 1500));
                yield return i;
            }
        }
    }
}
```

Which do I use, ConfigureAwait True or False?

The direct answer to this question is:

1. If you are writing code for the UI, use `ConfigureAwait(true)`.
2. If you are writing code in a library that will be shared, use `ConfigureAwait(false)`

In the next article, I am going to discuss [Asynchronous Disposable in C# 8](#) with Examples. Here, in this article, I try to explain **Asynchronous Streams in C# 8** with Examples. I hope you enjoy this Asynchronous Streams in C# with Examples article.